

# End-to-End

An Indian enterprise wants an assistant that answers questions from 200k internal docs. Fine-tuning costs ₹10+ lakh and goes stale. RAG ships in a week and stays fresh. The decision is not close.

How many internal docs does your last employer have? How much of that is indexed?

## WHAT'S INSIDE

- 01** RAG vs fine-tuning
- 02** The 5-stage pipeline
- 03** Chunking — the most under-engineered...
- 04** Evaluate retrieval like search
- 05** 40-line RAG — one file

WHY THIS LESSON MATTERS

# 200,000 internal docs. One AI assistant. Zero retraining.

---

WHY

## Real

A real reason this matters in industry today.

SCALE

## Today

A real reason this matters in industry today.

SPEED

## Now

A real reason this matters in industry today.

IMPACT

## Yours

A real reason this matters in industry today.

LEARNING OUTCOMES

# By the end of this lesson, you will be able to —

1

Why RAG beats fine-tuning for most enterprises.

2

The 5-stage RAG pipeline.

3

Design chunking given a corpus type.

4

Measure retrieval quality with precision@k + NDCG.

5

Build a RAG in 40 lines of Python.

AGENDA

# What we'll cover today

Short, sequenced parts. Each one builds on the last.

**01**

## RAG vs fine-tuning

Start with RAG. Fine-tune only after RAG has failed and you have an engineering diagnosis.

**02**

## The 5-stage pipeline

Chunk → Embed → Index → Retrieve  
→ Generate

**03**

## Chunking — the most...

400 tokens + 15% overlap

**04**

## Evaluate retrieval like...

precision@k, recall@k, NDCG

# 01

PART 1 OF 4

## RAG vs fine-tuning

---

Start with RAG. Fine-tune only after RAG has failed and you have an engineering diagnosis.

# RAG vs fine-tuning

Start with RAG. Fine-tune only after RAG has failed and you have an engineering diagnosis.

## DEFINITION

# RAG vs fine-tuning

Start with RAG. Fine-tune only after RAG has failed and you have an engineering diagnosis.

---

## EXAMPLE

*Real-world example: applies directly to Indian industry contexts.*

## CONCEPT 2 OF 4

# The 5-stage pipeline

1. Chunk — split docs into retrievable units.
2. Embed — convert chunks to vectors.
3. Index — store vectors in a vector DB.
4. Retrieve — embed the query; return top-k.
5. Generate — LLM answers using retrieved context.

PIPELINE

5 stages

Master one at a time; integrate at the end.

## KEY POINTS

- 1. Chunk — split docs into retrievable units
- 2. Embed — convert chunks to vectors
- 3. Index — store vectors in a vector DB
- 4. Retrieve — embed the query; return top-k

CONCEPT 3 OF 4

# Chunking — the most under-engineered step

Fixed-size: simple, narrative text.

Semantic: split on section / heading boundaries.

Overlap 10–20% prevents context loss at boundaries.

Parent-document: small child for retrieval; large parent for context.

DEFAULT

400 + 15%

Works for most Indian enterprise text.

DEFINITION

## Chunking — the most under-engineered...

400 tokens + 15% overlap

---

EXAMPLE

*Real-world example: applies directly to Indian industry contexts.*



## CONCEPT 4 OF 4

# Evaluate retrieval like search

precision@k: of the k returned, how many relevant?

recall@k: of all relevant, how many in k?

NDCG: discounted by rank position.

Hit-rate: at least one relevant in k — useful early.

BOOTSTRAP

50 hand-labelled pairs

> 50 hours of vibes.

## KEY POINTS

- precision@k: of the k returned, how many relevant?
- recall@k: of all relevant, how many in k?
- NDCG: discounted by rank position
- Hit-rate: at least one relevant in k — useful early

# 02

PART 2 OF 4

## Engage and reflect

---

Pause, talk, and connect what you just learned to your own work.

✦ THINK • PAIR •  
SHARE

ACTIVITY

# Index and query a 10-doc corpus

## PROMPT

*Apply the four concepts above to one real example from your own week.*

## HOW TO RUN IT

1

2

3

4

Reveal: instructor confirms the canonical answer.

REFLECT & APPLY

# Three questions before you close this lesson

Spend 60 seconds on each. Write the answers down — no scrolling, no shortcuts.

**Q1**

*Where in your org would RAG win today?*

**Q2**

*What chunking strategy fits your corpus?*

**Q3**

*How will you hand-label 50 pairs?*

# 03

PART 3 OF 4

## Knowledge check

---

Three short questions. One minute each. Honest answers only.

## RAG stands for:

A

Rapid AI Generation

B

Retrieval-Augmented Generation

C

Ranked Attention Generator

D

Recursive Agent Grid

***RAG stands for:***



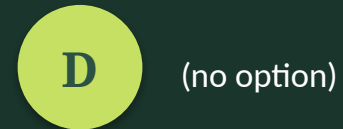
CORRECT ANSWER

**Retrieval-Augmented Generation**

WHY

Retrieval-Augmented Generation

## Default chunk size:





*Default chunk size:*



CORRECT ANSWER

D

WHY

The correct option matches the lesson's core principle.

## Fine-tune vs RAG default:

A

Fine-tune

B

RAG

C

Always fine-tune

D

Neither

## *Fine-tune vs RAG default:*



CORRECT ANSWER

**RAG**

### WHY

RAG first. Fine-tune only when RAG fails

# 04

PART 4 OF 4

## Apply and close

---

One thing to remember. One thing to ship this week.

SUMMARY

# Five sentences to remember

If you remember nothing else from this lesson, remember these five lines.

- 1 Pick the right tool — never the loudest one.
- 2 Practice the framework on a real example.
- 3 Watch for the three classic pitfalls.
- 4 Document your decision in one sentence.
- 5 Carry the discipline forward to the next lesson.

## WORKED EXAMPLE

# How this lesson plays out in one real Indian context

## THE SCENARIO

Start with RAG. Fine-tune only after RAG has failed and you have an engineering diagnosis.

## THE TAKEAWAY

*"Never mix them up."*

## WHAT TO NOTICE

1

RAG vs fine-tuning

2

The 5-stage pipeline

3

Chunking — the most...

4

Evaluate retrieval like...

# RAG over your notes

Pick your markdown/PDF notes. Build a RAG and ask 10 questions.

## DELIVERABLES

- Index with ChromaDB.
- 40-line Python pipeline.
- Measure precision@3.
- Post results.

## WHAT 'GOOD' LOOKS LIKE

*Deliverable: Repo URL + 10 (Q, A, retrieved-ids) examples.*

ONE THING TO REMEMBER

“

*"Never mix them up."*

— AI Learning Hub



## END OF LESSON 5

# What you can now do

Each one of these is now part of your working vocabulary.

- 1 Pick the right tool — never the loudest one.
- 2 Practice the framework on a real example.
- 3 Watch for the three classic pitfalls.
- 4 Document your decision in one sentence.
- 5 Carry the discipline forward to the next lesson.

NEXT → Lesson 4 — Vector databases, the engine under RAG.